

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 4 marzo 2010 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. Si consideri l'espressione regolare R seguente:

$$R = (a \mid c)^* (c (a \mid b))^+$$

Si risponda alle domande seguenti:

- (a) Utilizzando l'algoritmo di Berry e Sethi, si costruisca un automa deterministico A che riconosce il linguaggio definito da R .
 - (b) (facoltativa) Si mostri che R è ambigua, preferibilmente esibendo almeno un esempio di stringa ambigua e spiegando perché esso è tale.
-

Soluzione

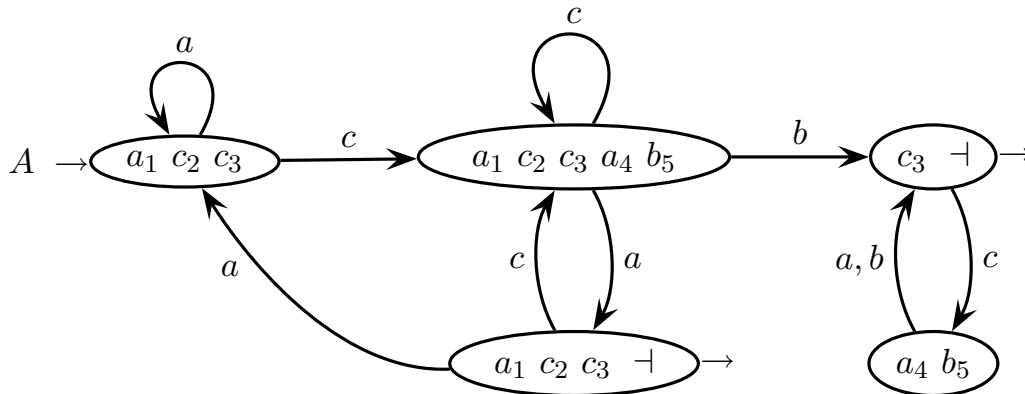
- (a) Per prima cosa si scrive la versione numerata R' dell'espressione R :

$$R' = (a_1 \mid c_2)^* (c_3 (a_4 \mid b_5))^+$$

Poi si compilano gli insiemi degli inizi e dei séguiti dell'espressione R' :

inizi	$a_1 \ c_2 \ c_3$
generatore	séguiti
a_1	$a_1 \ c_2 \ c_3$
c_2	$a_1 \ c_2 \ c_3$
c_3	$a_4 \ b_5$
a_4	$c_3 \ \dashv$
b_5	$c_3 \ \dashv$

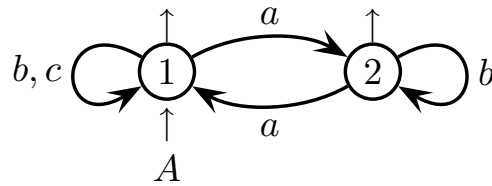
Da questi si ricava il grafo stato-transizione dell'automa A deterministico.



È piuttosto facile constatare che l'automa A (deterministico) è in forma minima.

- (b) L'espressione regolare R è ambigua perché le stringhe di tipo $(c a)^+$ (p. es. $c a$, $c a c a$, ecc) possono essere generate sia dalla sottoespressione $R_1 = (a \mid c)^*$ sia dalla sottoespressione $R_2 = (c (a \mid b))^+$. Sono dunque ambigue le stringhe di tipo $(c a)^n$ con $n > 1$. Naturalmente potrebbero esistere stringhe ambigue anche d'altro tipo (il lettore si può esercitare a trovarle).

2. Dato l'alfabeto $\Sigma = \{a, b, c\}$, si consideri l'automa deterministico A seguente:

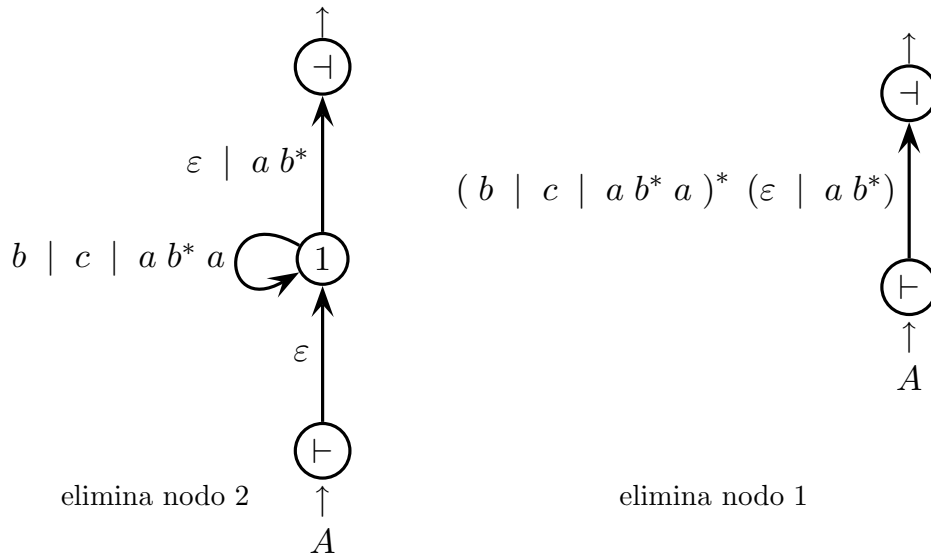
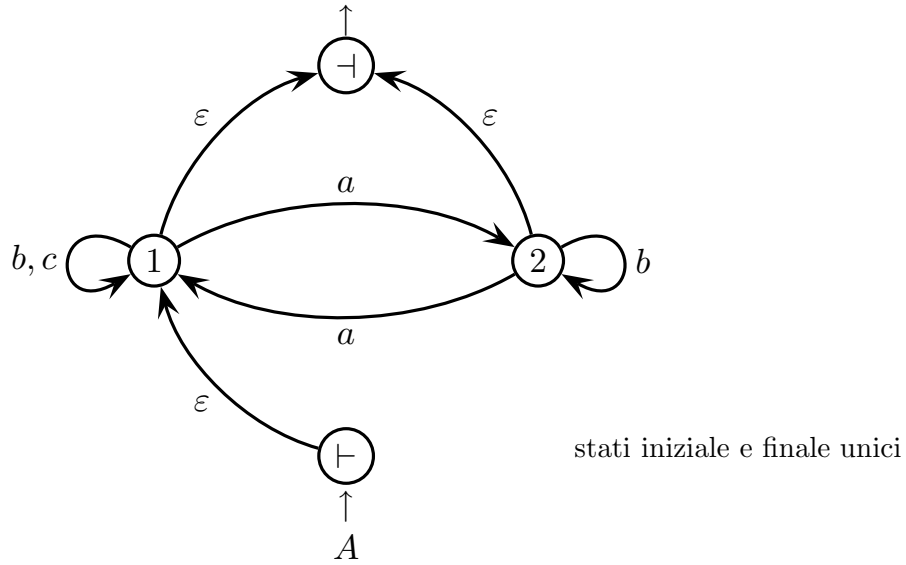


Si risponda alle domande seguenti:

- (a) Con un metodo a scelta, si ricavi l'espressione regolare R equivalente ad A .
 - (b) Si consideri il linguaggio $L' = L(A)^R$, riflesso del linguaggio riconosciuto da A , e se ne ricavi l'automa riconoscitore deterministico A' .
 - (c) (facoltativa) Se occorre, si minimizzi l'automa A' ottenuto precedentemente.
-

Soluzione

(a) All'automa A si applica il metodo di eliminazione dei nodi (Brzozowski):

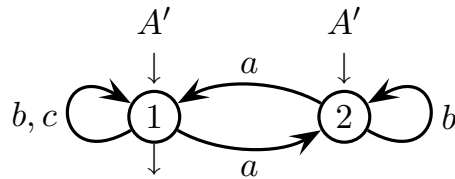


Ed ecco un'espressione regolare R equivalente all'automa A :

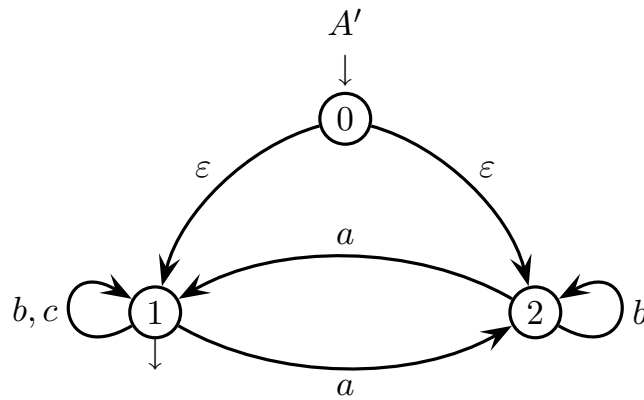
$$R = (b \mid c \mid a b^* a)^* (\varepsilon \mid a b^*)$$

Naturalmente per l'espressione sono possibili formulazioni diverse, per esempio secondo l'ordine di eliminazione dei nodi.

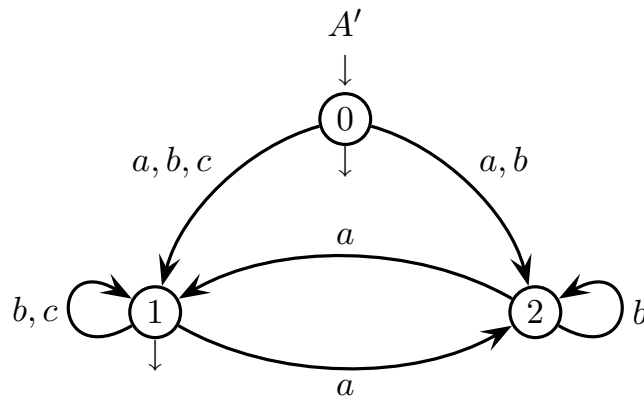
- (b) Per ottenere l'automa A' , va invertito l'orientamento degli archi e vanno scambiati gli stati iniziali e finali di A :



Così però A' è indeterministico poiché ha due stati iniziali, e va determinizzato. Prima si riduce a un solo stato iniziale mediante transizioni spontanee, così:



Poi si eliminano le transizioni spontanee, che qui vengono ricostruite come transizioni con input, e lo stato iniziale 0 diventa anche finale, così:



Infine si determinizza tramite la costruzione dei sottinsiemi.

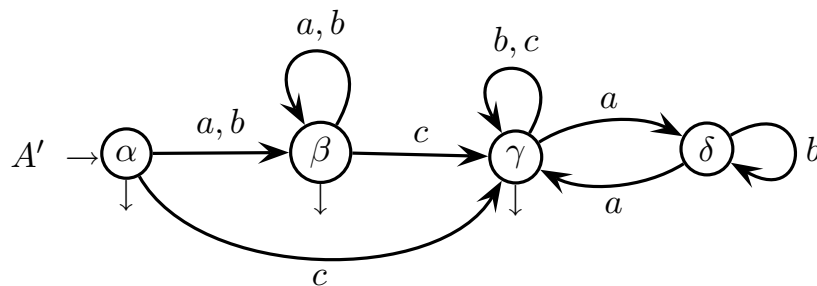
Ecco la tabella dei successori:

	a	b	c	finale
0	1 2	1 2	1	sì
1 2	1 2	1 2	1	sì
1	2	1	1	sì
2	1	2	—	no

Volendo si possono rinominare gli stati, così:

	a	b	c	finale
α	β	β	γ	sì
β	β	β	γ	sì
γ	δ	γ	γ	sì
δ	γ	δ	—	no

Questa è la tabella degli stati dell'automa A' deterministico. È facile constatare che l'automa è in forma ridotta, ossia tutti gli stati sono utili (raggiungibili e definiti). Volendo si può disegnare il grafo stato-transizione di A' . Eccolo:

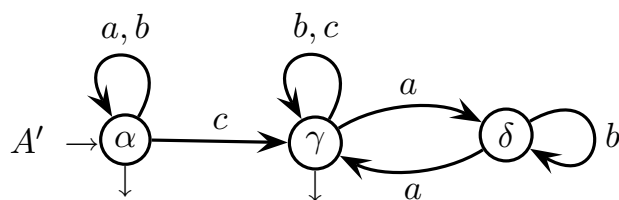


- (c) Si hanno solo tre possibili equivalenze tra stati: $\alpha \sim \beta$, $\alpha \sim \gamma$ e $\beta \sim \gamma$; infatti le altre coppie sarebbero tra stato finale e non finale. La prima coppia vale senz'altro perché le due prime righe sono identiche, mentre la seconda e la terza cadono nella colonna a dove si ha $\beta \not\sim \delta$ (β è finale e δ non finale).

Pertanto l'automa A' in forma minima è il seguente (identificando β con α):

	a	b	c	finale
α	α	α	γ	sì
γ	δ	γ	γ	sì
δ	γ	δ	—	no

Ecco il grafo stato-transizione dell'automa A' minimo:



Incidentalmente si nota che il grafo non è fortemente connesso, perché dai nodi γ e δ non si torna nel nodo α . Ciò comunque è un aspetto secondario.

2 Grammatiche libere e automi a pila 20%

1. Si consideri la grammatica G seguente (assioma S):

$$G \left\{ \begin{array}{l} S \rightarrow A C S \mid \varepsilon \\ A \rightarrow A b \mid a A b \mid a b \\ C \rightarrow b C \mid b C c \mid b c \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si dimostri che G è ambigua, mostrando la stringa ambigua più breve generata da essa, con gli alberi sintattici relativi.
 - (b) Si descriva, nel modo più chiaro e conciso possibile, il linguaggio generato da G .
 - (c) (facoltativa) Si definisca una grammatica G_N equivalente a G ma non ambigua, e si mostri qual è l'unico albero sintattico della stringa indicata al punto (a).
-

Soluzione

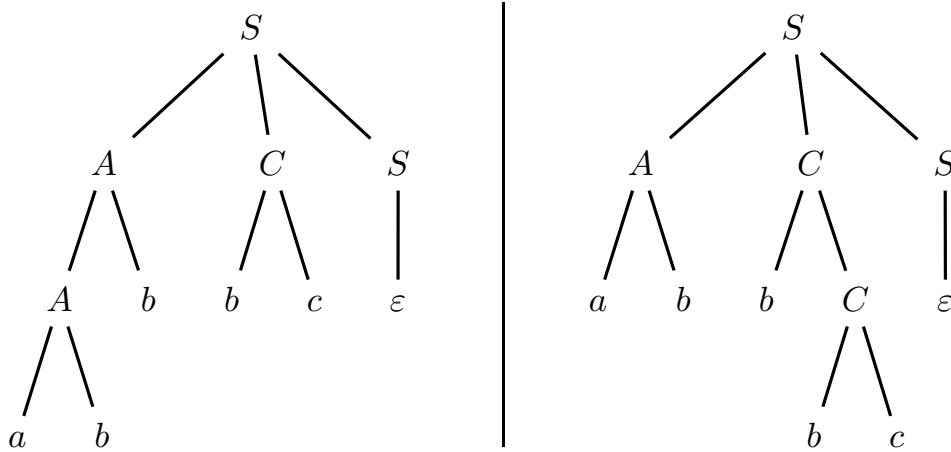
- (a) Tra le stringhe w del linguaggio $L(G)$, oltre a ε (che non è ambigua) le più corte e semplici sono le seguenti: $w = x y$, con $x = a^m b^n$ ($1 \leq m \leq n$) e $y = b^k c^h$ ($k \geq h \geq 1$); e i fattori concatenati x e y sono generati dai nonterminali A e C , rispettivamente. Così si ha:

$$w = a^m b^n b^k c^h = a^m b^m b^\alpha b^h c^h$$

dove $\alpha = (n - m) + (k - h) \geq 0$. Le α lettere b centrali, in eccesso rispetto al numero $m + h$ di lettere a e c , sono attribuibili sia a x sia a y (basta aggiustare gli esponenti n e k), e dunque sono generabili sia da A sia da C . Ecco pertanto la stringa ambigua più corta:

$$a b b b c \quad \text{ossia} \quad \underbrace{a b b}_A \underbrace{b c}_C \quad \text{oppure} \quad \underbrace{a b}_A \underbrace{b b c}_C$$

Ed ecco pertanto i due alberi sintattici della stringa $a b b b c$:



- (b) Riprendendo quanto detto prima, si vede che in generale le stringhe del linguaggio $L(G)$ sono ripetizioni dei fattori concatenati $x y$ per zero, una o più volte, dove ovviamente a ogni ripetizione x e y possono differire dai precedenti. Pertanto le stringhe $w_l \in L(G)$ ($l \geq 0$) hanno la struttura complessiva seguente:

$$w_l = \prod_{i=1}^l x_i y_i = \varepsilon |_{l=0}, x_1 y_1 |_{l=1}, x_1 y_1 x_2 y_2 |_{l=2}, \dots |_{l=\dots}$$

dove $l \geq 1$ e per ogni $1 \leq i \leq l$ si hanno $x_i = a^{m_i} b^{n_i}$ ($1 \leq m_i \leq n_i$) e $y_i = b^{k_i} c^{h_i}$ ($k_i \geq h_i \geq 1$), e dove come caso limite si pone $w_0 = \varepsilon$.

Ecco allora la descrizione formale (nel formalismo logico della teoria degli insiemi) del linguaggio generato dalla grammatica G :

$$L(G) = \left\{ \varepsilon, \prod_{i=1}^l a^{m_i} b^{n_i} b^{k_i} c^{h_i} \mid l \geq 1 \quad 1 \leq m_i \leq n_i \quad k_i \geq h_i \geq 1 \right\}$$

Chiaramente questo linguaggio non è regolare, dato che i fattori x_i e y_i sono notoriamente liberi ma non regolari.

Volendo si può trovare una descrizione formale di $L(G)$ più compatta:

$$L(G) = \left\{ \varepsilon, \prod_{i=1}^l a^{m_i} b^{s_i} c^{h_i} \mid l, m_i, h_i \geq 1 \quad s_i \geq m_i + h_i \right\}$$

Tuttavia non sembra altrettanto facile poi ricavarne una grammatica. Certamente essa esiste giacché il linguaggio è libero, ma la disuguaglianza tra i tre esponenti non la rende visibile immediatamente.

- (c) Riprendendo di nuovo quanto detto prima, si può osservare che la descrizione formale del linguaggio è ambigua, dato che la medesima stringa w_l ($l \geq 1$) è ottenibile in modi diversi aggiustando opportunamente gli esponenti n_i e k_i .

Si può però eliminare l'ambiguità vietando ai fattori x_i e y_i di generare le lettere b eccedenti, e riservando tale compito esclusivamente a un nuovo fattore b^* interposto tra i due. Ecco la nuova descrizione, non ambigua:

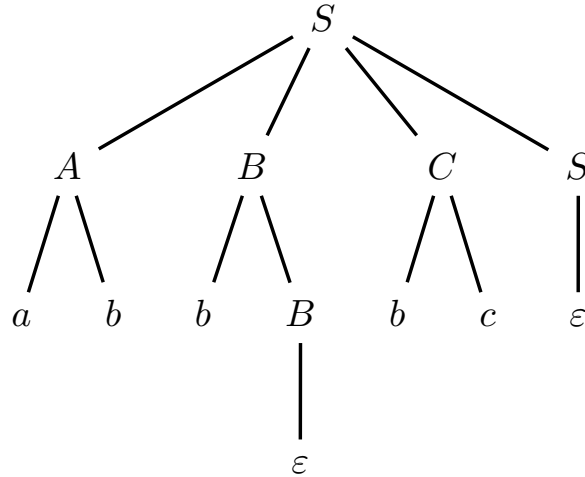
$$L(G) = \left\{ \varepsilon, \prod_{i=1}^l a^{m_i} b^{m_i} b^* b^{h_i} c^{h_i} \mid l, m_i, h_i \geq 1 \right\}$$

In termini grammaticali, per evitare l'ambiguità si può affidare la generazione delle lettere b eccedenti a un nuovo nonterminale B interposto tra i nonterminali A e C , i quali ora generano non ambiguamente solo nidi bilanciati $a b$ e $b c$, rispettivamente. Ecco pertanto la grammatica G_N non ambigua (assioma S), equivalente a G :

$$G_N \left\{ \begin{array}{l} S \rightarrow A B C S \mid \varepsilon \\ A \rightarrow a A b \mid a b \\ B \rightarrow b B \mid \varepsilon \\ C \rightarrow b C c \mid b c \end{array} \right.$$

dove le lettere b in eccedenza tra i nidi $a b$ e $b c$ consecutivi sono generate in modo unico dal nonterminale B interposto.

Ed ecco infine l'unico albero sintattico della stringa d'esempio $a b b b c$:



dove il nonterminale B genera la lettera b in eccesso tra i nidi $a b$ e $b c$.

2. Si considerino i tipi di *loop* (cicli) e gli altri costrutti sintattici illustrati tramite esempi nel listato seguente (tratto dal manuale di Modula 2):

```
1 MODULE LoopDemo;
  FROM Terminal2 IMPORT WriteString, WriteInt, WriteLn, WriteChar;
  CONST Where = 11;
  VAR   Index   : INTEGER;
        What    : INTEGER;
2       Letter  : CHAR;

3 BEGIN
  WriteString ("REPEAT loop      = ");
  Index := 0;
  REPEAT
    Index := Index + 1;
    WriteInt (Index, 5);
  UNTIL Index = 5;          (* This can be any BOOLEAN expression *)
  WriteLn;

  WriteString ("WHILE loop      = ");
  Index := 0;
  WHILE Index < 5 DO        (* This can be any BOOLEAN expression *)
    Index := Index + 1;
    WriteInt (Index, 5);
  END;
  WriteLn;

  WriteString ("First FOR loop  = ");
  FOR Index := 1 TO 5 DO
    WriteInt (Index, 5);
  END;
  WriteLn;

  WriteString ("Second FOR loop = ");
  FOR Index := 5 TO 25 BY 4 DO
    WriteInt (Index, 5);
  END;
  WriteLn;

  WriteString ("Third FOR loop  = ");
  FOR Index := 5 TO -35 BY -7 DO
    WriteInt (Index, 5);
  END;
  WriteLn;

  (* continua *)
```

```

What := 16;
FOR Index := (What - 21) TO (What * 2) BY Where DO
    WriteString ("Fourth FOR loop = ");
    WriteInt (Index, 5);
    WriteLn;
END;

Index := 1;
LOOP
    WriteString ("In the EXIT loop ");
    WriteInt (Index, 5);
    IF Index = 5 THEN
        WriteLn;
        EXIT;
    END;
    WriteString (" We are still in the loop.");
    WriteLn;
    Index := Index + 1;
END;

END LoopDemo.

```

Si scriva una grammatica G , non ambigua e in forma EBNF, per generare i loop e i costrutti sintattici esemplificati nel listato (si trascurino i commenti), come meglio precisato di seguito.

- (a) Si diano le regole per generare l'intestazione e la struttura generale del modulo, e le dichiarazioni di sottoprogramma, costante e variabile illustrate nell'intervallo di linee da 1 a 2.
 - (b) Si aggiungano le regole per generare gli statement esecutivi illustrati dalla linea 3 in poi, ossia: loop (dei vari tipi), assignment, procedure (o function) invocation, if e exit. Per semplicità si suppone che le espressioni aritmetiche a_expr e booleane b_expr siano già definite.
 - (c) (facoltativa) Si completi la definizione delle sole espressioni booleane b_expr . aggiungendo le regole opportune. Le espressioni booleane possono contenere gli operatori OR , AND , NOT e $<$ (elencati in ordine crescente di precedenza), le espressioni aritmetiche a_expr e le parentesi.
-

Soluzione

(a) Ecco le regole base di G per intestazione e dichiarazioni (assioma PROG):

$$G \left\{ \begin{array}{l} \langle \text{PROG} \rangle \rightarrow \text{'MODULE'} \langle \text{ID} \rangle \text{' ; ' } \langle \text{DECL} \rangle \langle \text{EXEC} \rangle \langle \text{ID} \rangle \text{' . ' } \\ \langle \text{DECL} \rangle \rightarrow \langle \text{EXTRN} \rangle^* [\langle \text{CONST} \rangle] [\langle \text{VAR} \rangle] \\ \langle \text{EXEC} \rangle \rightarrow \text{'BEGIN'} \langle \text{BODY} \rangle \text{'END'} \\ \langle \text{EXTRN} \rangle \rightarrow \text{'FROM'} \langle \text{ID} \rangle \text{'IMPORT'} \langle \text{ID} \rangle (\text{' , ' } \langle \text{ID} \rangle)^* \text{' ; ' } \\ \langle \text{CONST} \rangle \rightarrow \text{'CONST'} (\langle \text{ID} \rangle \text{' = ' } \langle \text{INTEGER} \rangle \text{' ; ' })^+ \\ \langle \text{VAR} \rangle \rightarrow \text{'VAR'} (\langle \text{ID} \rangle \text{' : ' } (\text{'CHAR'} \mid \text{'INTEGER'}) \text{' ; ' })^+ \\ \langle \text{BODY} \rangle \rightarrow (\langle \text{STAT} \rangle \text{' ; ' })^+ \end{array} \right.$$

(b) Ecco le regole da aggiungere a G per espandere lo statement (assioma STAT):

$$G \left\{ \begin{array}{l} \langle \text{STAT} \rangle \rightarrow \langle \text{ASS} \rangle \mid \langle \text{CALL} \rangle \mid \\ \quad \langle \text{WHI} \rangle \mid \langle \text{REP} \rangle \mid \langle \text{FOR} \rangle \mid \langle \text{LOOP} \rangle \mid \\ \quad \langle \text{IF} \rangle \mid \text{'EXIT'} \\ \langle \text{ASS} \rangle \rightarrow \langle \text{ID} \rangle \text{' := ' } \langle \text{A_EXP} \rangle \\ \langle \text{CALL} \rangle \rightarrow \langle \text{ID} \rangle [\text{' (' } \langle \text{PAR} \rangle (\text{' , ' } \langle \text{PAR} \rangle)^* \text{') ' }] \\ \langle \text{PAR} \rangle \rightarrow \langle \text{STRING} \rangle \mid \langle \text{A_EXP} \rangle \\ \langle \text{WHI} \rangle \rightarrow \text{'WHILE'} \langle \text{B_EXP} \rangle \text{'DO'} \langle \text{BODY} \rangle \text{'END'} \\ \langle \text{REP} \rangle \rightarrow \text{'REPEAT'} \langle \text{BODY} \rangle \text{'UNTIL'} \langle \text{B_EXP} \rangle \text{'END'} \\ \langle \text{FOR} \rangle \rightarrow \text{'FOR'} \langle \text{ASS} \rangle \text{'TO'} \langle \text{A_EXP} \rangle [\text{'BY'} \langle \text{A_EXP} \rangle] \\ \quad \text{'DO'} \langle \text{BODY} \rangle \text{'END'} \\ \langle \text{LOOP} \rangle \rightarrow \text{'LOOP'} \langle \text{BODY} \rangle \text{'END'} \\ \langle \text{IF} \rangle \rightarrow \text{'IF'} \langle \text{B_EXP} \rangle \text{'THEN'} \langle \text{BODY} \rangle \text{'END'} \end{array} \right.$$

(c) Ecco le regole da aggiungere a G per espandere le espr. bool. (assioma B_EXP):

$$G \left\{ \begin{array}{l} \langle \text{B_EXP} \rangle \rightarrow \langle \text{TERM} \rangle (\text{'OR'} \langle \text{TERM} \rangle)^* \\ \langle \text{TERM} \rangle \rightarrow \langle \text{FACT} \rangle (\text{'AND'} \langle \text{FACT} \rangle)^* \\ \langle \text{FACT} \rangle \rightarrow [\text{'NOT'}] \langle \text{N_FACT} \rangle \\ \langle \text{N_FACT} \rangle \rightarrow \text{' (' } \langle \text{B_EXP} \rangle \text{') ' } \mid \langle \text{A_EXP} \rangle \text{' < ' } \langle \text{A_EXP} \rangle \end{array} \right.$$

I nonterminali A_EXP, STRING, INTEGER e ID, qui non sono espansi.

La grammatica G data riproduce quanto si vede negli esempi forniti sopra, senza generalizzare. Naturalmente gli esempi illustrano costrutti separati, ma si deve ammettere di poterli innestare ossia di avere ricorsione, come in qualunque linguaggio di programmazione. Ecco alcune precisazioni sulla struttura di G :

- si ammettono più clausole **FROM**, facoltative e non vuote, per importare procedure e funzioni da più librerie com'è consentito nella maggior parte dei linguaggi di programmazione; gli esempi ne mostrano una sola, ma chiaramente sarebbe una limitazione un po' eccessiva
- c'è una sola sezione **CONST**, facoltativa e non vuota, con costanti
- c'è una sola sezione **VAR**, facoltativa e non vuota, con variabili
- procedure e funzioni hanno lista di parametri, facoltativa e non vuota
- nel corpo della parte esecutiva del modulo ci dev'essere almeno uno statement, e in generale nel corpo di ogni costrutto

Il resto della grammatica G è piuttosto ovvio. I nomi delle classi sintattiche (non-terminali) sono autoesplicativi e aiutano a comprenderne il significato. Volendo, si potrebbero raggruppare nella medesima dichiarazione le costanti con valore identico e le variabili dello stesso tipo, separandone gl'identificatori con virgola, com'è consentito in numerosi linguaggi di programmazione; e per completezza si potrebbero perfino elencare più librerie nella medesima clausola **FROM**. Così:

$$\langle \text{CONST} \rangle \rightarrow \text{'CONST' } (\langle \text{ID} \rangle (\text{' ,' } \langle \text{ID} \rangle)^* \text{'=' } \langle \text{INTEGER} \rangle \text{' ;' })^+$$

Similmente per la clausola **FROM** e la sezione **VAR**. Sarebbero generalizzazioni del tutto ragionevoli, sebbene non siano mostrate negli esempi forniti sopra.

Quanto alle espressioni booleane, le regole rispettive sono modellate sulla ben nota grammatica EBNF delle espressioni aritmetiche con parentesi, dando agli operatori **OR**, **AND** e **NOT** la precedenza richiesta. L'operatore **NOT** è trattato come segno meno unario. L'operatore relazionale **<** ha precedenza massima.

La grammatica G proposta è modulare e pertanto ragionevolmente corretta, ed è costituita da strutture notoriamente non ambigue, pertanto ragionevolmente non è ambigua. Beninteso ci sono altre formulazioni possibili, altrettanto valide. Come del resto ci possono essere piccole varianti nel modo d'interpretare gli esempi e di dedurre la formulazione della grammatica.

3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica G seguente (assioma S):

$$G \left\{ \begin{array}{l} S \rightarrow a S X \mid \varepsilon \\ X \rightarrow b X c \mid \varepsilon \end{array} \right.$$

Si risponda alle domande seguenti:

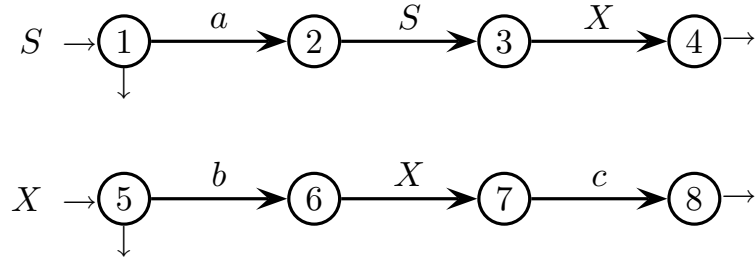
- (a) Si rappresenti G come rete di automi finiti e si mostri la successione di stati che l'analizzatore sintattico discendente attraversa riconoscendo la stringa seguente:

$a b c$

- (b) Si trovino tutti gl'insiemi guida per $k = 1$ e si dica se G è di tipo $LL(1)$.
 - (c) (facoltativa) Se occorre, si dica se G è di tipo $LL(2)$.
-

Soluzione

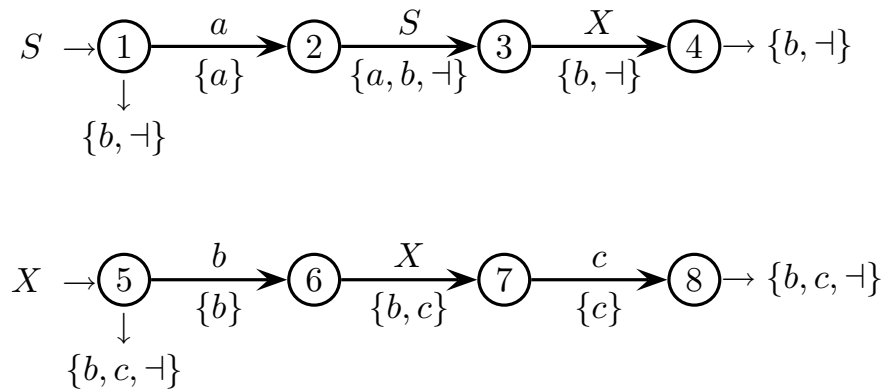
(a) Ecco gli automi finiti che rappresentano la grammatica G :



La successione di stati attraversati dall'analizzatore discendente è la seguente:

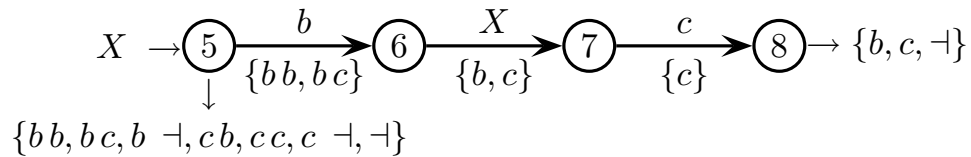
$$1 \xrightarrow{a} 2 \rightarrow 1 \rightarrow 3 \rightarrow 5 \xrightarrow{b} 6 \rightarrow 5 \rightarrow 7 \xrightarrow{c} 8 \rightarrow 4$$

(b) Ecco tutti gl'insiemi guida per $k = 1$:



La grammatica G non è $LL(1)$ nello stato 5.

(c) Ecco gl'insiemi guida per $k = 2$ (solo dove serve):



La grammatica G non è $LL(2)$ nello stato 5.

In conclusione, nello stato 5 la grammatica G non è né $LL(1)$ né $LL(2)$.

2. Sono date le due grammatiche G_1 e G_2 seguenti (assiomi S_1 e S_2):

$$G_1 \left\{ \begin{array}{l} S_1 \rightarrow B \\ B \rightarrow B b \mid a A b \\ A \rightarrow a A b \mid \varepsilon \end{array} \right. \quad G_2 \left\{ \begin{array}{l} S_2 \rightarrow C \\ C \rightarrow b C \mid b D \\ D \rightarrow D c \mid \varepsilon \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si verifichi se G_1 e G_2 soddisfano la condizione $LR(1)$.
 - (b) (facoltativa) Per il linguaggio $L = L_1 \cdot L_2$ (concatenamento di $L_1 = L(G_1)$ e $L_2 = L(G_2)$), si scriva una grammatica G che soddisfa la condizione $LR(1)$.
-

Soluzione

- (a) Entrambe le grammatiche G_1 e G_2 soddisfano la condizione $LR(1)$. Si mostra la parte più significativa dell'automa pilota di G_1 :

$S_1 \rightarrow \bullet B$	$\neg$		$B \rightarrow a \bullet A b$	$\neg b$		$A \rightarrow a \bullet A b$	b
$B \rightarrow \bullet B b$	$\neg b$	$\xrightarrow{a}$	$A \rightarrow \bullet a A b$	b	$\xrightarrow{a}$	$A \rightarrow \bullet a A b$	b
$B \rightarrow \bullet a A b$	$\neg b$		$A \rightarrow \bullet \varepsilon = \varepsilon \bullet$	b		$A \rightarrow \bullet \varepsilon = \varepsilon \bullet$	b

e del pilota di G_2 :

$S_2 \rightarrow \bullet C$	$\neg$		$C \rightarrow b \bullet C$	$\neg$
$C \rightarrow \bullet b C$	$\neg$	$\xrightarrow{b}$	$C \rightarrow b \bullet D$	$\neg$
$C \rightarrow \bullet b D$	$\neg$		$C \rightarrow \bullet b C$	$\neg$
			$C \rightarrow \bullet b D$	$\neg$
			$D \rightarrow \bullet D c$	$\neg c$
			$D \rightarrow \bullet \varepsilon = \varepsilon \bullet$	$\neg c$

dove si vede che le due candidate di riduzione nulle $A, D \rightarrow \varepsilon \bullet$ non sono conflittuali con gli spostamenti, mentre tutte le altre candidate di riduzione finiscono individualmente in macrostati separati e pertanto non danno problema.

- (b) La grammatica G' più ovvia per il linguaggio $L = L_1 \cdot L_2$ si ottiene aggiungendo a G_1 e G_2 la regola assiomatica $S' \rightarrow S_1 S_2$ (o più concisamente la $S' \rightarrow B C$ che rende inutili le due vecchie regole assiomatiche). Ma tale G' soffre di ambiguità di concatenamento, che la rende non $LR(k)$. Però osservando che:

$$L = \overbrace{\{a^n b^n b^* \mid n \geq 1\}}^{L_1} \cdot \overbrace{b^+ c^*}^{L_2} = \{a^n b^n \mid n \geq 1\} b^+ c^*$$

si scrive facilmente la grammatica G seguente (assioma S):

$$G \left\{ \begin{array}{l} S \rightarrow B C \\ B \rightarrow a A b \\ A \rightarrow a A b \mid \varepsilon \\ C \rightarrow b C \mid b D \\ D \rightarrow D c \mid \varepsilon \end{array} \right.$$

Intuitivamente, questa grammatica G è $LR(1)$. Infatti, la derivazione destra prima espande il nonterminale C , che genera il fattore $b^+ c^*$, e poi espande il nonterminale B , che genera il fattore $a^n b^n$ ($n \geq 1$) a sinistra del fattore $b^+ c^*$. L'analizzatore sintattico muove da sinistra verso destra e pertanto prima scandisce il fattore context-free $a^n b^n$, impilando n simboli mentre legge gli n caratteri a e spilandoli mentre legge i primi n caratteri b , e poi scandisce il fattore regolare $b^+ c^*$, senza più usare la pila in modo significativo bensì solamente gli stati. È evidente che l'analizzatore così descritto produce la derivazione destra in ordine inverso, e che dunque è di tipo ascendente. Inoltre esso è chiaramente deterministico. Ne viene che la grammatica G è di tipo $LR(1)$. Però essa non è di tipo $LR(0)$, giacché contiene regole nulle.

4 Traduzione e analisi semantica 20%

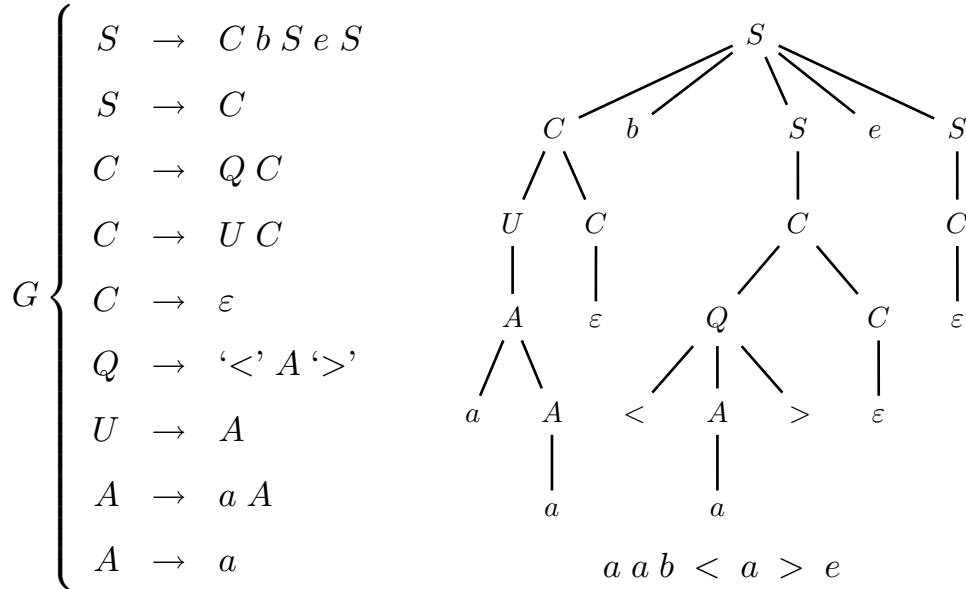
1. Si consideri un messaggio strutturato come una frase di Dyck di alfabeto $\{b, e\}$. Esso, oltre a detti simboli, può contenere una o più stringhe di alfabeto a , eventualmente racchiuse tra i caratteri ' $<$ ' e ' $>$ ', le parentesi angolari o uncinate. Ma le lettere a , se non sono racchiuse tra parentesi angolari, vanno cancellate dal traduttore.

Esempio di traduzione τ :

$$x = b \overbrace{< a a >}^{\text{conserva}} e b \overbrace{a a a}^{\text{cancella}} \overbrace{< a >}^{\text{conserva}} b e \overbrace{a}^{\text{cancella}} e \overbrace{< a a a >}^{\text{conserva}}$$

$$\tau(x) = b < a a > e b < a > b e e < a a a >$$

L'alfabeto completo è dunque $\{a, b, e, <, >\}$. Si fornisce una grammatica G del linguaggio sorgente (assioma S), con a lato un piccolo albero sintattico d'esempio:



Se è necessario od opportuno per progettare il traduttore che calcola τ , è permesso modificare la grammatica G .

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica di traduzione (o se si preferisce uno schema di traduzione) per calcolare la traduzione τ descritta sopra.
- (b) Si disegnino gli alberi sintattici sorgente e immagine per la stringa x data nell'esempio sopra.
- (c) (facoltativa) Si dica, con opportune motivazioni, se la traduzione τ è calcolabile deterministicamente.

Soluzione

- (a) Per realizzare la traduzione τ , si modifica la grammatica G . L'idea di fondo è quella di differenziare il nonterminale A in due nonterminali A_Q e A_U , che generano le lettere a racchiuse tra parentesi angolari ed esterne a esse, rispettivamente (i pedici Q e U stanno per “quoted” ossia “tra parentesi” e “unquoted”). Nella grammatica destinazione il nonterminale A_U non genera lettere a e così si realizza la traduzione richiesta. Ecco lo schema sintattico della traduzione τ .

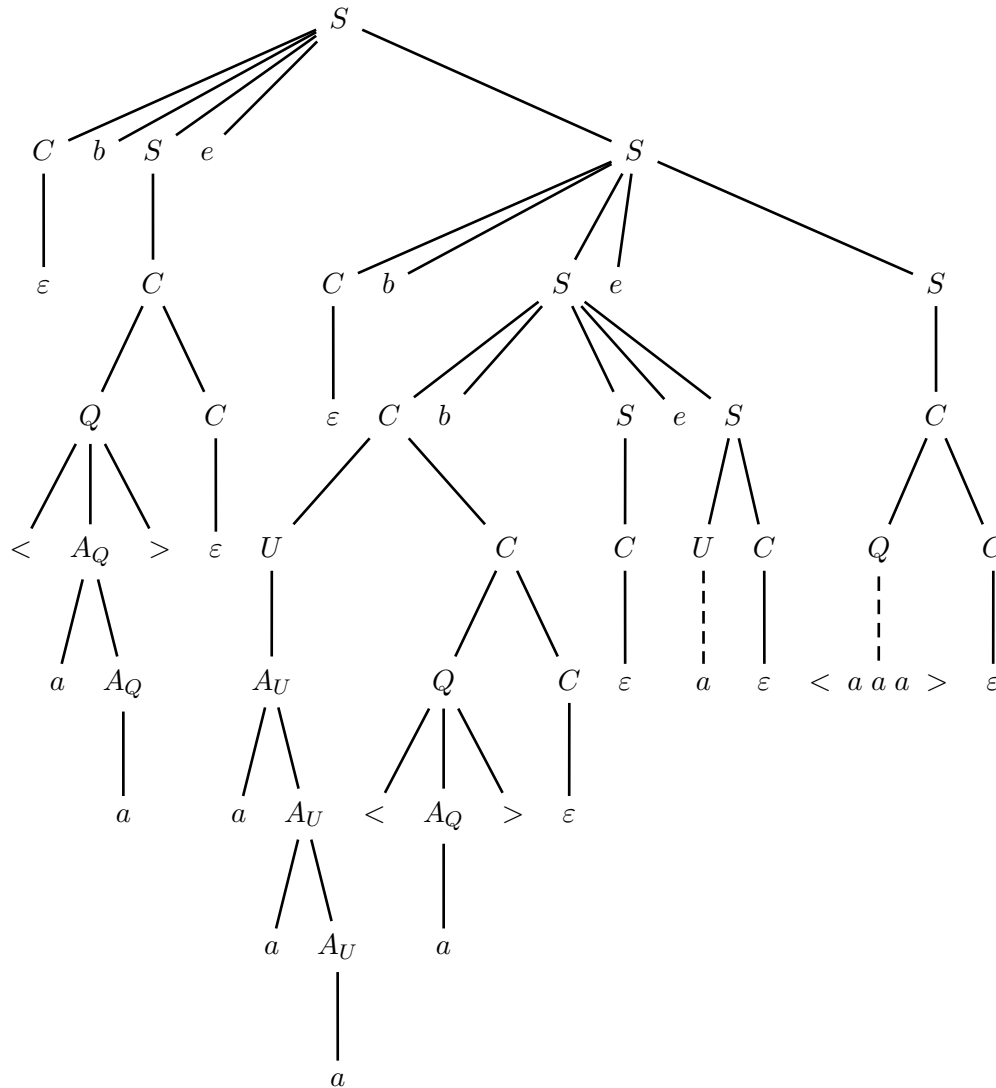
gram. G_s sorgente (ass. S)	gram. G_d destinazione (ass. S)
$G_s \left\{ \begin{array}{l} S \rightarrow C b S e S \\ S \rightarrow C \\ C \rightarrow Q C \\ C \rightarrow U C \\ C \rightarrow \varepsilon \\ Q \rightarrow '<' A_Q '>' \\ U \rightarrow A_U \\ A_Q \rightarrow a A_Q \\ A_Q \rightarrow a \\ A_U \rightarrow a A_U \\ A_U \rightarrow a \end{array} \right.$	$G_d \left\{ \begin{array}{l} S \rightarrow C b S e S \\ S \rightarrow C \\ C \rightarrow Q C \\ C \rightarrow U C \\ C \rightarrow \varepsilon \\ Q \rightarrow '<' A_Q '>' \\ U \rightarrow A_U \\ A_Q \rightarrow a A_Q \\ A_Q \rightarrow a \\ A_U \rightarrow A_U \\ A_U \rightarrow \varepsilon \end{array} \right.$

Ecco invece la versione di τ come grammatica di traduzione G_t (assioma S):

$$G_t \left\{ \begin{array}{ll} S \rightarrow C b \{ b \} S e \{ e \} S & A_Q \rightarrow a \{ a \} A_Q \\ S \rightarrow C & A_Q \rightarrow a \{ a \} \\ C \rightarrow Q C & A_U \rightarrow a A_U \\ C \rightarrow U C & A_U \rightarrow a \\ C \rightarrow \varepsilon & \\ Q \rightarrow '<' \{ '<' \} A_Q '>' \{ '>' \} & \\ U \rightarrow A_U & \end{array} \right.$$

Beninteso ci possono essere altre soluzioni, ottenute diversamente.

(b) Ecco l'albero sintattico sorgente (un po' semplificato) della stringa x :



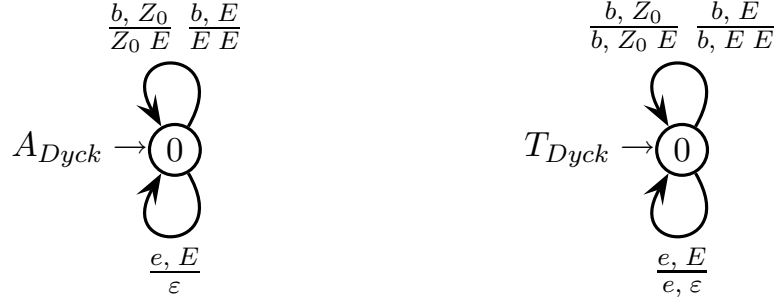
L'albero sintattico destinazione o immagine è simile ed è lasciato al lettore.

(c) Un modo per trovare se la traduzione τ è deterministica, consiste nel verificare se la grammatica sorgente G_s è deterministica di tipo $LL(1)$. Si vede immediatamente che essa non è ambigua. Tuttavia si vede pure subito che le prime due regole hanno insiemi guida sovrapposti:

$$\begin{array}{l|l} S \rightarrow C b S e S & \text{guida} \ni a \\ S \rightarrow C & \text{guida} \ni a \end{array}$$

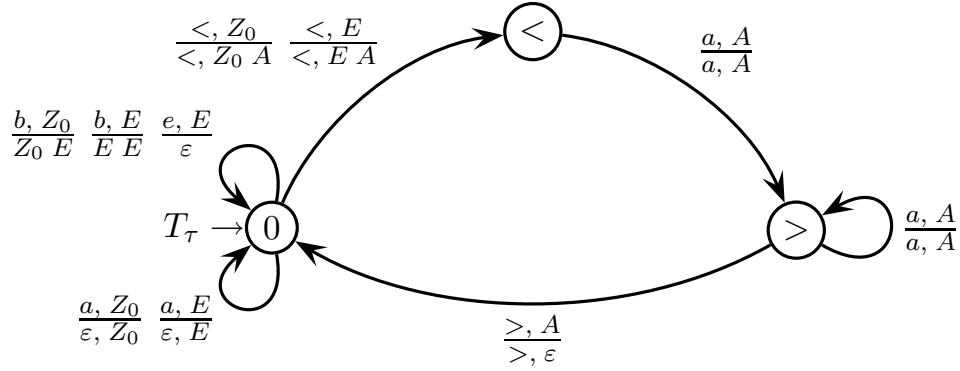
Anche allungando la prospezione, la sovrapposizione resta. Dunque non si può costruire il trasduttore sintattico deterministico di tipo discendente, per questa grammatica di traduzione G_t .

Ciò non significa che un trasduttore a pila deterministico non esista, sebbene non sintattico ossia non modellato sulla grammatica G_t . Si sa che il linguaggio di Dyck è deterministico e che un puro riconoscitore deterministico A_{Dyck} , con un solo stato e a pila vuota, è il seguente (per l'alfabeto di due lettere $\{b, e\}$):



È immediato trasformare l'automa riconoscitore A_{Dyck} in trasduttore T_{Dyck} , pure deterministico con un solo stato e a pila vuota, che traduce il linguaggio di Dyck in sé stesso, come si vede sopra (a destra).

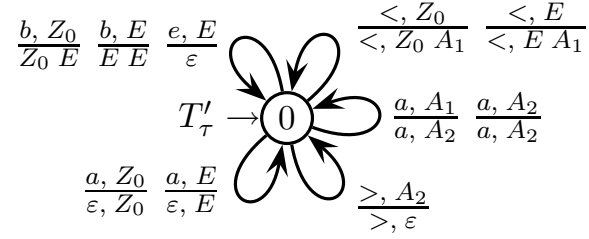
Ora è facile modificare il trasduttore T_{Dyck} così da ottenere la traduzione τ . Basta usare gli stati finiti, senza ricorrere alla pila, per riconoscere la coppia di parentesi angolari conservando le lettere a contenute (almeno una), altrimenti cancellandole. Ecco il trasduttore T_τ (con tre stati e a pila vuota):



Conformemente alla grammatica sorgente G_s , tra le parentesi angolari ci dev'essere almeno una lettera a e le parentesi non sono annidate. Il simbolo di pila A viene impilato e successivamente spilato riconoscendo la parentesi $<$ e $>$, rispettivamente. Esso serve soltanto per non avere pila vuota mentre è in corso la traduzione della coppia di parentesi, al fine di evitare di riconoscere quando la coppia non è ancora chiusa. Si vede subito che il trasduttore T_τ è deterministico. Pertanto la traduzione τ è deterministica, giacché esiste un trasduttore a pila deterministico che la realizza. Tuttavia il trasduttore T_τ non è un analizzatore sintattico (né discendente né ascendente) per la grammatica di traduzione G_t , poiché da un calcolo di T_τ non si ricostruisce una derivazione di G_t .

Di conseguenza, una grammatica di traduzione deterministica (o uno schema di traduzione), di tipo discendente e che realizzi τ , deve avere una struttura diversa da quella della grammatica sorgente G_s . Sarebbe possibile ottenere questa nuova grammatica trasformando il trasduttore deterministico a pila T_τ dato sopra.

Ecco come fare. Prima, lavorando sui simboli di pila e differenziando A in A_1 e A_2 , si riducono gli stati a uno solo pur mantenendo il determinismo:



I simboli A_1 e A_2 codificano sulla cima della pila i due stati finiti eliminati $<$ e $>$, rispettivamente, traducendo la coppia di parentesi angolari e il contenuto. Poi, essendo il trasduttore T'_τ a un solo stato, è immediato trasformarlo in grammatica di traduzione G'_t (assioma S) mappando transizioni su regole, così:

	regola	guida
G'_t	$S \rightarrow b \{ b \} E S$	b
	$S \rightarrow '<' \{ '<' \} A_1 S$	$<$
	$S \rightarrow a S$	a
	$S \rightarrow \varepsilon$	$\neg$
	$A_1 \rightarrow a \{ a \} A_2$	a
	$A_2 \rightarrow a \{ a \} A_2$	a
	$A_2 \rightarrow '>' \{ '>' \}$	$>$
	$E \rightarrow b \{ b \} E E$	b
	$E \rightarrow '<' \{ '<' \} A_1 E$	$<$
	$E \rightarrow a E$	a
	$E \rightarrow e \{ e \}$	e

Si vede subito che la grammatica di traduzione G'_t è deterministica di tipo $LL(1)$, dato che gli insiemi guida per $k = 1$ di regole alternative sono disgiunti. Strutturalmente, G'_t è diversa da G_t , ma esse sono equivalenti (in senso debole). Infine, in alternativa si potrebbe verificare se la grammatica sorgente G_s sia deterministica di tipo $LR(1)$ (certamente non $LR(0)$ perché il linguaggio sorgente ha prefissi). Se fosse così, sarebbe possibile mettere in forma postfissa la grammatica di traduzione G_t e dunque costruire un trasduttore sintattico deterministico di tipo ascendente che realizza la traduzione τ . Un punto di partenza per tale verifica potrebbe essere quello di esaminare la grammatica G data nel testo: se essa fosse di tipo $LR(1)$, ragionevolmente anche la grammatica sorgente modificata G_s potrebbe esserlo.

Si lascia al lettore di esaminare l'alternativa. Intuitivamente però, la grammatica G è davvero deterministica $LR(1)$. Infatti si può osservare quanto segue:

- il linguaggio generato dal nonterminale C è regolare di alfabeto $\{a, <, >\}$, avendosi $L(C) = (a \mid < a^+ >)^*$, e le regole che espandono C non danno luogo a derivazioni autoinclusive, pertanto ragionevolmente esse sono $LR(1)$
- le due regole assiomatiche sono quelle ben note, non ambigue e di tipo $LR(1)$, che generano il linguaggio di Dyck di alfabeto $\{b, e\}$ intercalato liberamente da stringhe del linguaggio $L(C)$
- gli alfabeti $\{a, <, >\}$ e $\{b, e\}$ sono disgiunti e da C non si deriva l'assioma S , pertanto le regole che espandono S e quelle che espandono C finiscono in macrostati LR separati

Se ne conclude che l'analizzatore sintattico ascendente è una sorta di composizione modulare di due gruppi di macrostati separati e indipendentemente senza conflitti, e che pertanto non dovrebbe avere conflitti neppure globalmente. Questa non è una vera dimostrazione, bensì una considerazione incoraggiante.

2. La grammatica G seguente (assioma S) genera espressioni parentetiche non vuote.

$$G \left\{ \begin{array}{l} S \rightarrow A \\ A \rightarrow A A \\ A \rightarrow '(A)' \\ A \rightarrow '(,)' \end{array} \right.$$

A partire da essa, si vuole costruire una grammatica con attributi che permetta di stabilire se un'espressione parentetica è uniformemente profonda, nel senso che tutti i nidi di parentesi più interni si trovano allo stesso livello di annidamento.

Esempi di nidi uniformemente profondi:

$() \quad (()) \quad ()() \quad ((())) \quad ((()))((())())$

Controesempi:

$()(()) \quad ((())())$

Si risponda alle domande seguenti:

- (a) Si definisca una tale grammatica con attributi, preferibilmente facente uso solo dei due attributi p e b seguenti, entrambi sinistri (sintetizzati):
- $p \in Integer$ profondità del nido di parentesi
 - $b \in Boolean$ il nido di parentesi è uniformemente profondo

Per rispondere si usino gli schemi predisposti alle pagine seguenti.

- (b) Si dica se la grammatica con attributi proposta è a una sola passata (one sweep), motivando adeguatamente la risposta.
- (c) (facoltativa) Si scriva la procedura semantica associata al nonterminale A e che valuta gli attributi p e b .
-

attributi da usare per la grammatica

tipo	nome	(non)terminali	dominio	significato
sin	p	A	intero	profondità del nido di parentesi
sin	b	S, A	booleano	il nido di parentesi è uniformemente profondo

sintassi

semantica

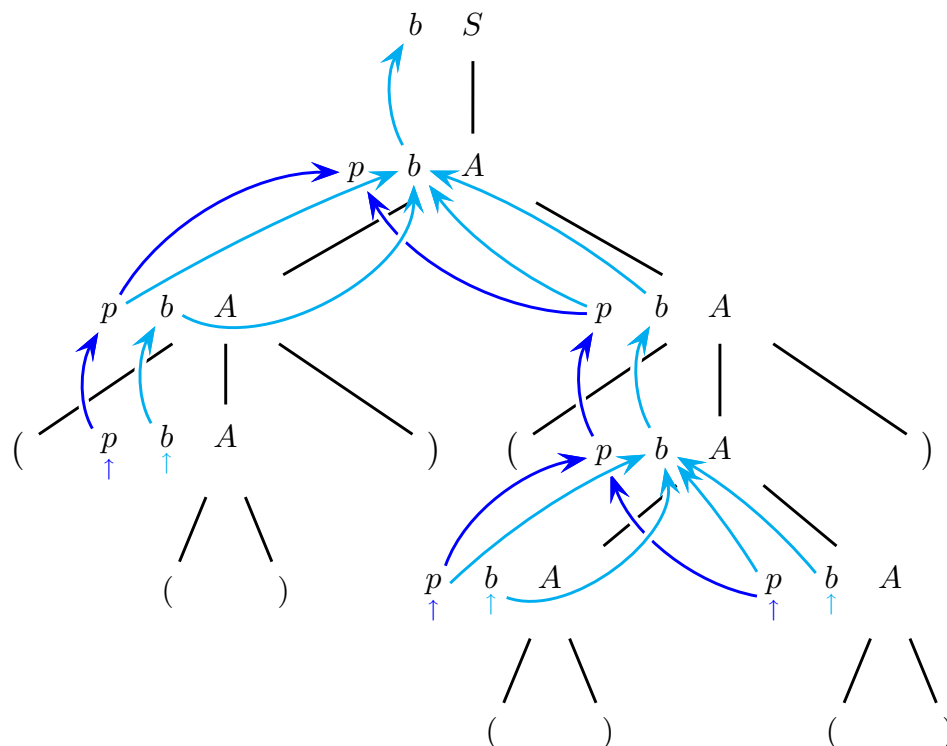
$S_0 \rightarrow A_1$	
$A_0 \rightarrow A_1 A_2$	
$A_0 \rightarrow '(A_1)'$	
$A_0 \rightarrow '()'$	

Soluzione

- (a) I due attributi p e b proposti bastano per definire la grammatica richiesta. L'idea di fondo è che con p si calcola la profondità del nido generato dal nonterminale A , e che ogniqualvolta si hanno due nidi concatenati, ossia $A A$, si verifica che le profondità rispettive siano uguali, conservando quest'informazione e propagandola verso la radice dell'albero con b . Ecco dunque le funzioni semantiche:

sintassi	semantica
$S_0 \rightarrow A_1$	$b_0 = b_1$
$A_0 \rightarrow A_1 A_2$	$p_0 = \max(p_1, p_2)$ $b_0 = b_1 \wedge b_2 \wedge (p_1 == p_2)$
$A_0 \rightarrow '(' A_1 ')'$	$p_0 = p_1 + 1$ $b_0 = b_1$
$A_0 \rightarrow '(', ')'$	$p_0 = 1$ $b_0 = true$

Ed ecco un piccolo albero sintattico decorato per la stringa $(()) (()) (())$.



La stringa non è ambigua, ma G lo è, e in generale ci sono più alberi sintattici.

- (b) La grammatica con attributi è a una sola passata (one sweep) perché possiede solo attributi sintetizzati. Si noti peraltro che il supporto sintattico è ambiguo, a causa della regola $A \rightarrow A A$ che presenta ricorsione bilatera. Pertanto la sintassi non è analizzabile deterministicamente, né con il metodo *LL* né con quello *LR*. Di conseguenza l'analizzatore semantico non è integrabile con quello sintattico (che non esiste).
- (c) I due attributi sinistri p e b sono parametri in uscita. Dato che la grammatica non ha attributi destri, non ci sono parametri in ingresso (tranne ovviamente la radice T del (sotto)albero da valutare). Ecco la procedura semantica del nonterminale A :

```

procedure  $A$  ( in  $T$ : tree, out  $p$ : integer, out  $b$ : boolean )
var
     $p_1, p_2$ : integer
     $b_1, b_2$ : boolean
begin
    case rule of
         $A \rightarrow A A$ : begin
            call  $A$  (  $T \rightarrow A_1, p_1, b_1$  )
            call  $A$  (  $T \rightarrow A_2, p_2, b_2$  )
             $p = \max( p_1, p_2 )$ 
             $b = b_1$  and  $b_2$  and (  $p_1 == p_2$  )
        end
         $A \rightarrow '(' A ')'$ : begin
            call  $A$  (  $T \rightarrow A_1, p_1, b_1$  )
             $p = p_1 + 1$ 
             $b = b_1$ 
        end
         $A \rightarrow '(' ' )'$ : begin
             $p = 1$ 
             $b = \text{true}$ 
        end
        otherwise error
    end case
end procedure

```

Può darsi che il codice della procedura sia ottimizzabile, in vari modi.